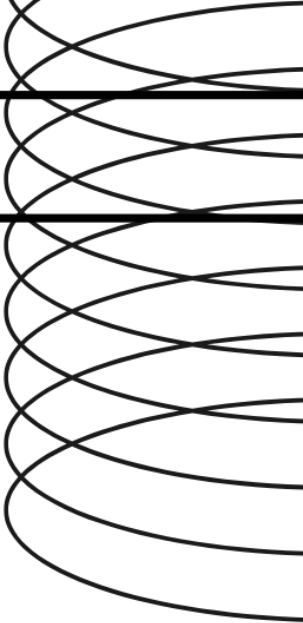
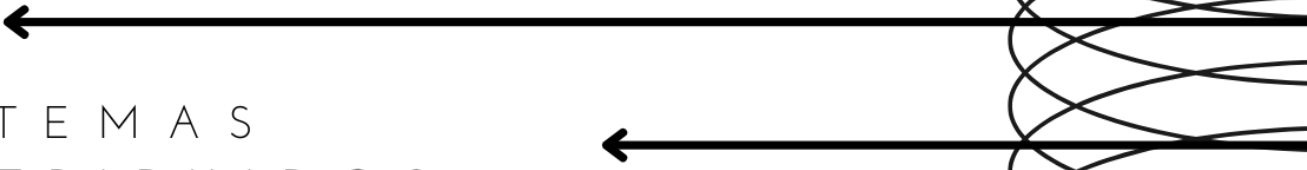




SISTEMAS
DISTRIBUIDOS

Práctica 3.

Procesos multiconectados y sincronizados.

MICHELLE ANGUIANO RODEA

ESCOM

PROFESOR CHADWICK CARRETO
ARELLANO

GRUPO: 7CM6



Instituto
politécnico
nacional

ESCOM

1. Antecedentes (Introducción)

En los sistemas distribuidos modernos, no basta con permitir la comunicación entre un servidor y un cliente de forma individual. Las aplicaciones reales deben ser capaces de manejar múltiples clientes de manera simultánea, garantizando al mismo tiempo la sincronización cuando acceden a recursos compartidos.

Un servidor multiconectado es aquel que acepta y gestiona varias conexiones concurrentes, asignando un canal independiente a cada cliente. Para lograrlo se utilizan sockets TCP, que proporcionan un canal confiable de comunicación bidireccional. Sin embargo, cuando varios clientes intentan acceder a la misma información o recurso, surge la necesidad de aplicar técnicas de sincronización que eviten problemas como:

- Condiciones de carrera: cuando dos o más procesos modifican simultáneamente un recurso compartido, generando resultados inconsistentes.
- Bloqueos indefinidos (deadlocks): cuando varios procesos esperan indefinidamente por un recurso.
- Injusticia en la atención: cuando algunos clientes monopolizan el acceso al servidor y otros quedan en espera.

La combinación de multiprocesamiento con hilos y sincronización mediante semáforos o secciones críticas permite construir servidores robustos y escalables, que forman la base de aplicaciones distribuidas como servicios web, sistemas de mensajería instantánea, videojuegos en línea y bases de datos multiusuario.

2. Problemática

El problema a resolver en esta práctica fue implementar un sistema que cumpliera con las siguientes características:

1. Diseñar un servidor multiconectado capaz de aceptar múltiples clientes concurrentes.
2. Establecer un mecanismo de sincronización que garantice el acceso ordenado a recursos compartidos.
3. Implementar una cola de peticiones que atienda las solicitudes de acuerdo con criterios definidos:
 - Prioridad de la petición.
 - Importancia del recurso solicitado.

- Tiempo de llegada.
- 4. Verificar que la comunicación sea confiable y que los clientes reciban las respuestas procesadas de manera correcta.

En aplicaciones reales, este tipo de problemas es equivalente a la gestión de solicitudes en un servidor web, donde cada petición (HTTP request) debe ser atendida siguiendo reglas de prioridad, evitando la saturación o pérdida de datos.

3. Solución

La solución consistió en diseñar una arquitectura cliente-servidor con las siguientes características:

- Servidor (Server.java)
 - Escucha en un puerto definido.
 - Acepta conexiones entrantes y crea un hilo independiente (ClientHandler) para cada cliente.
 - Inserta las solicitudes en una cola de peticiones compartida.
- Manejador de clientes (ClientHandler.java)
 - Recibe las peticiones enviadas por el cliente.
 - Les asigna atributos como prioridad, importancia y tiempo de llegada.
 - Inserta las solicitudes en la cola compartida.
- Cola de peticiones (RequestQueue.java)
 - Implementada como una PriorityBlockingQueue, que ordena las solicitudes según las reglas establecidas (prioridad, importancia y antigüedad).
- Despachador (Dispatcher.java)
 - Extrae peticiones de la cola en orden.
 - Procesa cada solicitud dentro de una sección crítica protegida por un semáforo.
 - Envía la respuesta correspondiente al cliente.
- Cliente (Client.java)
 - Se conecta al servidor.

- Envía solicitudes con un formato específico (PRIORIDAD;IMPORTANCIA;MENSAJE).
- Espera y recibe la respuesta procesada por el servidor.

Este diseño asegura que:

- Cada cliente es atendido concurrentemente.
- No existen condiciones de carrera al acceder a la cola.
- Se respeta un criterio justo y ordenado en la atención de las solicitudes.

4. Implementación

El sistema fue implementado en Java y consta de las siguientes clases principales:

1. Request.java → Define el objeto petición con atributos de prioridad, importancia, tiempo de llegada y mensaje.

```
import java.time.Instant;
import java.util.concurrent.atomic.AtomicLong;

public class Request {
    private static final AtomicLong SEQ = new AtomicLong(initialValue:0);

    public final long seq;           // rompe empates
    public final long arrivalNanos; // tiempo de llegada
    public final int importance      // 1 = alta, 2 = media, 3 = baja
    public final int importance;     // 1..10 (10 muy importante)
    public final String payload;     // mensaje
    public final ClientHandler origin; // para responder

    public Request(int priority, int importance, String payload, ClientHandler origin) {
        this.seq = SEQ.getAndIncrement();
        this.arrivalNanos = System.nanoTime();
        this.priority = priority;
        this.importance = importance;
        this.payload = payload;
        this.origin = origin;
    }

    @Override
    public String toString() {
        return "Request{prio=" + priority + ", imp=" + importance +
            ", at=" + Instant.now() + ", from=" + origin.getClientId() +
            ", msg=" + payload + "'}";
    }
}
```

2. RequestQueue.java → Cola priorizada que ordena las peticiones según las reglas definidas.

```

RequestQueue.java 7 ...
1  import java.util.Comparator;
2  import java.util.concurrent.PriorityBlockingQueue;
3
4  public class RequestQueue {
5      // Regla: primero prioridad (1 mejor), Luego mayor importancia, Luego tiempo de Llegada.
6      private final PriorityBlockingQueue<Request> queue = new PriorityBlockingQueue<>(  
7          initialCapacity:100,  
8          Comparator  
9              .comparingInt((Request r) -> r.priority)           // menor primero  
10             .thenComparing((Request r) -> -r.importance)       // mayor primero  
11             .thenComparingLong(r -> r.arrivalNanos)             // más antiguo primero  
12             .thenComparingLong(r -> r.seq)                       // rompe empates  
13         );
14
15     public void put(Request r) { queue.put(r); }
16     public Request take() throws InterruptedException { return queue.take(); }
17     public int size() { return queue.size(); }
18 }

```

3. ClientHandler.java → Hilo que recibe peticiones de un cliente y las inserta en la cola.

```

import java.io.*;
import java.net.Socket;
import java.util.UUID;

public class ClientHandler implements Runnable {
    private final Socket socket;
    private final RequestQueue sharedQueue;
    private final String clientId;

    public ClientHandler(Socket socket, RequestQueue queue) {
        this.socket = socket;
        this.sharedQueue = queue;
        this.clientId = socket.getRemoteSocketAddress() + "-" + UUID.randomUUID().toString().substring(beginIndex:0, endIndex:6);
    }

    public String getClientId() { return clientId; }

    @Override
    public void run() {
        System.out.println("[+] Cliente conectado: " + clientId);
        try (BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
             PrintWriter out = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()), autoFlush:true)) {

            out.println(x:"Conectado al servidor. Formato: PRIORIDAD;IMPORTANCIA;MENSAJE");
            out.println(x:"Ejemplo: 1;9;hola");

            String line;
            while ((line = in.readLine()) != null) {
                line = line.trim();

                if (line.equalsIgnoreCase(anotherString:"exit") || line.equalsIgnoreCase(anotherString:"quit")) break;

                int prio = 2, imp = 5;
                String msg = line;

                try {
                    // Formato: pri:msg (todo lo demás se toma como mensaje)
                    String[] parts = line.split(regex:";", limit:3);
                    if (parts.length >= 2) {
                        prio = Integer.parseInt(parts[0].trim());
                        imp = Integer.parseInt(parts[1].trim());
                        msg = parts.length == 3 ? parts[2] : "";
                    }
                } catch (Exception ignore) {}

                Request r = new Request(prio, imp, msg, this);
                sharedQueue.put(r);
                out.println("Recibida -> " + r.toString());
            }
        } catch (IOException e) {
            System.err.println("[ " + clientId + " ] Error IO: " + e.getMessage());
        } finally {
            try { socket.close(); } catch (IOException ignore) {}
            System.out.println("[-] Cliente desconectado: " + clientId);
        }
    }
}

```

4. Dispatcher.java → Hilo encargado de extraer las peticiones y procesarlas con sincronización.

```
import java.util.concurrent.Semaphore;

public class Dispatcher implements Runnable {
    private final RequestQueue queue;
    // Recurso critico simulado con 1 permiso (se podría subir a N para "N servicios").
    private final Semaphore criticalSection = new Semaphore(permits:1, fair:true);

    public Dispatcher(RequestQueue queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        System.out.println(x:"[*] Dispatcher iniciado");
        while (true) {
            try {
                Request r = queue.take(); // espera bloqueante
                criticalSection.acquire(); // sincronización: un request a la vez en sección crítica

                try {
                    // ----- sección crítica: "procesamiento del servicio" -----
                    System.out.println("[PROC] " + r);
                    // Simulación de trabajo proporcional a la importancia (más imp => más rápido)
                    long workMillis = Math.max(a:100, 800 - (r.importance * 60L));
                    Thread.sleep(workMillis);

                    String reply = "[OK] Atendido -> prio=" + r.priority +
                        ", imp=" + r.importance +
                        ", cliente=" + r.origin.getClientId() +
                        ", msg=\"\" + r.payload + "\"";

                    r.origin.sendResponse(reply);
                } finally {
                    criticalSection.release();
                }
            } catch (InterruptedException ie) {
                System.out.println(x:"[*] Dispatcher detenido.");
                Thread.currentThread().interrupt();
                break;
            }
        }
    }
}
```

5. Server.java → Clase principal que inicia el servidor, acepta clientes y lanza hilos de atención.

```

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Server {
    public static final int PORT = 8080;

    public static void main(String[] args) {
        RequestQueue queue = new RequestQueue();

        // Iniciar 1 (o más) despachadores que consumen de la cola
        Thread dispatcher = new Thread(new Dispatcher(queue), "dispatcher-1");
        dispatcher.start();

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Servidor escuchando en puerto " + PORT + " ...");
            while (true) {
                Socket clientSocket = serverSocket.accept();
                Thread t = new Thread(new ClientHandler(clientSocket, queue));
                t.start();
            }
        } catch (IOException e) {
            System.err.println("Error en servidor: " + e.getMessage());
        } finally {
            dispatcher.interrupt();
        }
    }
}

```

6. Client.java → Programa cliente que envía solicitudes al servidor y muestra las respuestas recibidas.

```

import java.io.*;
import java.net.Socket;
import java.util.Random;

public class Client {
    public static void main(String[] args) throws Exception {
        String host = (args.length > 0) ? args[0] : "127.0.0.1";
        int port = (args.length > 1) ? Integer.parseInt(args[1]) : 8080;

        try (Socket socket = new Socket(host, port);
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            BufferedReader stdin = new BufferedReader(new InputStreamReader(System.in));
            PrintWriter out = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()), true)) {

            // Mostrar banner del servidor
            for (int i = 0; i < 2; i++) System.out.println(in.readLine());

            System.out.println("Escribe líneas con formato: PRIORIDAD;IMPORTANCIA;MENSAJE");
            System.out.println("Ejemplo: 1;9;hola | escribe 'exit' para salir\n");

            // Hilo Lector (muestra respuestas asíncronas)
            Thread reader = new Thread(() -> {
                try {
                    String s;
                    while ((s = in.readLine()) != null) {
                        System.out.println("[SRV] " + s);
                    }
                } catch (IOException ignore) {}
            });
        }
    }
}

```

```

        reader.setDaemon(on:true);
        reader.start();

        // Enviar algunas peticiones demo si no hay teclado (o el usuario prefiere)
        if (System.console() == null) {
            Random rnd = new Random();
            for (int i = 0; i < 5; i++) {
                int prio = 1 + rnd.nextInt(bound:3);
                int imp = 1 + rnd.nextInt(bound:10);
                out.println(prio + ";" + imp + ";hola-" + i);
                Thread.sleep(millis:200);
            }
            Thread.sleep(millis:2000);
            out.println(x:"exit");
        } else {
            String line;
            while ((line = stdin.readLine()) != null) {
                out.println(line);
                if (line.equalsIgnoreCase(anotherString:"exit")) break;
            }
        }
    }
}

```

El uso de hilos (Thread) permite atender múltiples clientes en paralelo, mientras que los semáforos (Semaphore) aseguran que el servidor procese una petición a la vez dentro de la sección crítica.

5. Resultados

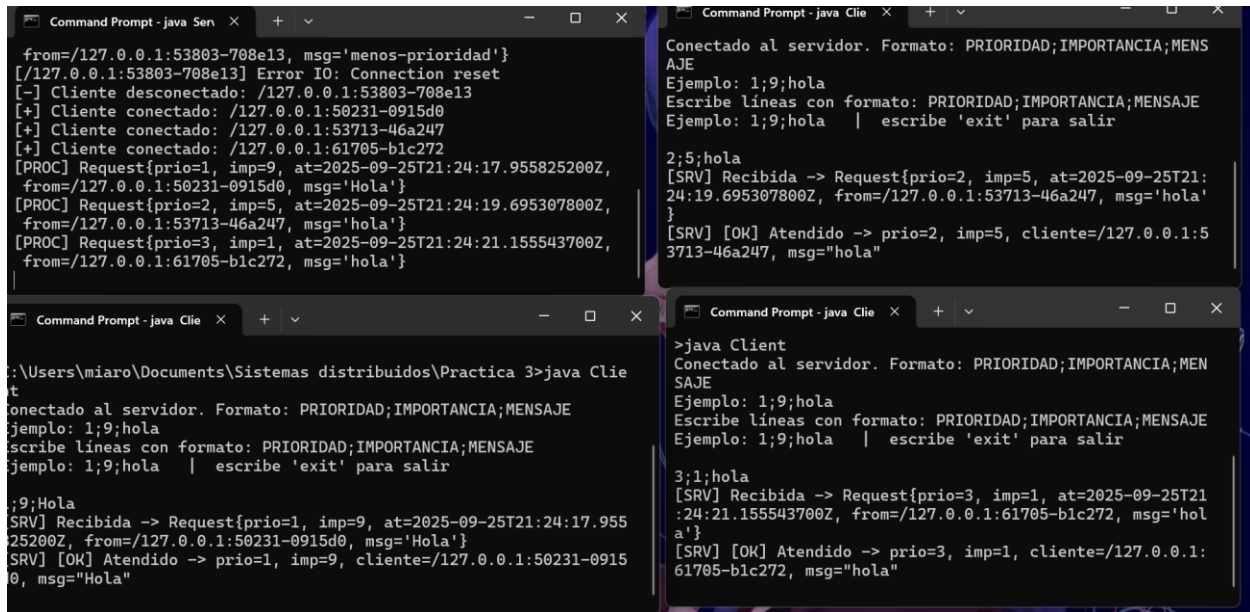
Durante la ejecución del sistema se obtuvieron los siguientes resultados:

- El servidor se mantuvo en escucha en el puerto definido (8080).
- Se conectaron múltiples clientes de manera concurrente.
- Cada cliente pudo enviar múltiples solicitudes sin bloquear a los demás.
- La cola de peticiones procesó correctamente las solicitudes, respetando el criterio de prioridad e importancia.
- Los clientes recibieron la respuesta de sus solicitudes de manera independiente y sin errores.
- No se presentaron condiciones de carrera ni bloqueos indefinidos.

Ejemplo de ejecución:

1. Cliente A envía: 3;1;Solicitud baja.
2. Cliente B envía: 1;9;Solicitud alta.
3. Cliente C envía: 2;5;Solicitud media.

El servidor procesó primero la petición del Cliente B, luego la del Cliente C y finalmente la del Cliente A, demostrando que la prioridad y la importancia se aplicaron correctamente.



```
from=/127.0.0.1:53803-708e13, msg='menos-prioridad'}
[/127.0.0.1:53803-708e13] Error IO: Connection reset
[-] Cliente desconectado: /127.0.0.1:53803-708e13
[+] Cliente conectado: /127.0.0.1:50231-0915d0
[+] Cliente conectado: /127.0.0.1:53713-46a247
[+] Cliente conectado: /127.0.0.1:61705-b1c272
[PROC] Request{prio=1, imp=9, at=2025-09-25T21:24:17.955825200Z,
from=/127.0.0.1:50231-0915d0, msg='Hola'}
[PROC] Request{prio=2, imp=5, at=2025-09-25T21:24:19.695307800Z,
from=/127.0.0.1:53713-46a247, msg='hola'}
[PROC] Request{prio=3, imp=1, at=2025-09-25T21:24:21.155543700Z,
from=/127.0.0.1:61705-b1c272, msg='hola'}

Conectado al servidor. Formato: PRIORIDAD;IMPORTANCIA;MENSAJE
Ejemplo: 1;9;hola
Escribe líneas con formato: PRIORIDAD;IMPORTANCIA;MENSAJE
Ejemplo: 1;9;hola | escribe 'exit' para salir

2;5;hola
[SRV] Recibida -> Request{prio=2, imp=5, at=2025-09-25T21:24:19.695307800Z, from=/127.0.0.1:53713-46a247, msg='hola'}
[SRV] [OK] Atendido -> prio=2, imp=5, cliente=/127.0.0.1:53713-46a247, msg="hola"

C:\Users\miaro\Documents\Sistemas distribuidos\Practica 3>java Client
Conectado al servidor. Formato: PRIORIDAD;IMPORTANCIA;MENSAJE
Ejemplo: 1;9;hola
Escribe líneas con formato: PRIORIDAD;IMPORTANCIA;MENSAJE
Ejemplo: 1;9;hola | escribe 'exit' para salir

3;1;hola
[SRV] Recibida -> Request{prio=3, imp=1, at=2025-09-25T21:24:21.155543700Z, from=/127.0.0.1:61705-b1c272, msg='hola'}
[SRV] [OK] Atendido -> prio=3, imp=1, cliente=/127.0.0.1:61705-b1c272, msg="hola"

C:\Users\miaro\Documents\Sistemas distribuidos\Practica 3>java Server
[*] Dispatcher iniciado
Servidor escuchando en puerto 8080 ...
```

6. Conclusión

La práctica permitió comprender y aplicar conceptos de multiprocesamiento y sincronización en un entorno distribuido.

Se concluye que:

1. Un servidor multiconectado requiere el uso de hilos para atender múltiples clientes simultáneamente.
2. La sincronización con semáforos es esencial para evitar condiciones de carrera y garantizar la consistencia en los datos.
3. El uso de una cola de peticiones priorizada asegura un tratamiento justo y ordenado de las solicitudes.
4. Este modelo constituye la base de sistemas distribuidos más avanzados como servidores web, bases de datos multiusuario y plataformas de comunicación en tiempo real.

7. Referencias

- Oracle. (n.d.). *Lesson: Concurrency (The Java™ Tutorials > Essential Classes > Concurrency)*. Recuperado de:
<https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- GeeksforGeeks. (n.d.). *Java Socket Programming*. Recuperado de:
<https://www.geeksforgeeks.org/socket-programming-in-java/>
- Baeldung. (n.d.). *A Guide to Java Semaphores*. Recuperado de:
<https://www.baeldung.com/java-semaphore>